DBMS Interview Guide

Basics

Q1. What is a database?

Answer:
A **database is an organized collection of data that can be stored, accessed, and managed efficiently**.

For example, instead of storing student information in separate files, we can store it in a database and easily search, update, or delete student records.

Example:
A college database might store:

Student_ID	Name	Course	Marks
101	Rahul	MCA	85
102	Priya	MCA	91

Q2. What is the difference between DBMS and RDBMS?

Answer:
DBMS is software used to store and manage data.

RDBMS is a type of DBMS that stores data in **tables and maintains relationships between those tables**.

RDBMS generally follows relational concepts such as primary keys, foreign keys, and constraints.

Example:

In a college system:

Students

Student_ID
Name
Course_ID

Courses

Course_ID
Course_Name

The Course_ID can connect the two tables. This is a typical **RDBMS** approach.

Examples of RDBMS include MySQL, PostgreSQL, Oracle, and SQL Server.

Q3. What is a table?

Answer:
A **table is a structure in a relational database that stores data in rows and columns**.

- **Row** → one record
- **Column** → one attribute/property

Example:

Students

ID	Name	Age
1	Rahul	22
2	Priya	23

Here, each student is a **row**, while ID, Name, and Age are **columns**.

Q4. What is a primary key?

Answer:
A **primary key is a column or combination of columns that uniquely identifies each row in a table**.

A primary key:

- Must be unique.
- Cannot contain NULL.

Example:

Student_ID	Name
101	Rahul
102	Priya

Student_ID can be the primary key because every student has a unique ID.

Real-world example:
Your college registration number uniquely identifies you among students.

Q5. What is a foreign key?

Answer:
A **foreign key is a column that refers to the primary key of another table**. It is used to establish a relationship between tables.

Example:

Students
Student_ID | Name | Course_ID
101 | Rahul| 10
102 | Priya| 20
Courses
Course_ID | Course_Name
10 | MCA
20 | MBA

Here, Students.Course_ID is a **foreign key** referring to Courses.Course_ID.

Real-world example:
A student belongs to a particular course. The Course_ID connects the student to that course.

Q6. What is the difference between Primary Key and Unique Key?

Answer:

Both ensure that values are unique, but there are some differences.

Primary Key	Unique Key
Uniquely identifies a row	Ensures values are unique
Cannot contain NULL	Can contain one NULL value (varies by DB)
Only one per table	Multiple per table

Example:

Student_ID → Primary Key

Email → Unique Key

Every student has a unique ID, and their email should also be unique.

Q7. What is a candidate key?

Answer:
A **candidate key is a column or combination of columns that can uniquely identify each row and is eligible to become the primary key**.

A table can have multiple candidate keys, but we choose one of them as the primary key.

Example:

```
Student_ID | Email           | Name
-----------|-----------------|------
101        | rahul@gmail.com | Rahul
102        | priya@gmail.com | Priya
```

Both Student_ID and Email can uniquely identify a student.

So:

Candidate Keys:
→ Student_ID
→ Email

If we choose Student_ID as the primary key, Email remains an alternate candidate key.

Q8. What is a composite key?

Answer:
A **composite key is a key made up of two or more columns that together uniquely identify a row**.

Example:

Suppose we have:

```
Student_ID | Course_ID | Semester
-----------|-----------|---------
101        | CS101     | 1
101        | DBMS      | 1
102        | CS101     | 1
```

Neither Student_ID nor Course_ID alone is necessarily unique.

But:

(Student_ID, Course_ID)

together can uniquely identify the enrollment.

That's a **composite key**.

SQL

Q9. What is the difference between WHERE and HAVING?

Answer:

WHERE filters **individual rows before grouping**.

HAVING filters **groups after GROUP BY**.

Example:

```
SELECT department, COUNT(*)
FROM employees
WHERE salary > 50000
GROUP BY department
HAVING COUNT(*) > 5;
```

Here:

WHERE → removes employees whose salary <= 50000
GROUP BY → creates groups by department
HAVING → keeps only departments with more than 5 employees

Q10. What is GROUP BY?

Answer:
GROUP BY is used to **group rows that have the same value in one or more columns**, usually so we can apply aggregate functions.

Example:

```
SELECT department, COUNT(*)
FROM employees
GROUP BY department;
```

Result could be:

Department	Count
IT	20
HR	8
Finance	12

It groups employees by department and counts them.

Real-world example:
A company wants to know how many employees work in each department.

Q11. What are the different types of JOINs?

Answer:
The commonly used SQL joins are:

1. **INNER JOIN** → matching rows from both tables.
2. **LEFT JOIN** → all rows from the left table + matching rows from the right.
3. **RIGHT JOIN** → all rows from the right table + matching rows from the left.
4. **FULL OUTER JOIN** → all rows from both tables, matching where possible.
5. **CROSS JOIN** → every row from one table combined with every row from another.
6. **SELF JOIN** → a table joined with itself.

Example:

Suppose:

Students
Student_ID | Name | Course_ID

Courses
Course_ID | Course_Name

We can join them using:

```
SELECT s.Name, c.Course_Name
FROM Students s
JOIN Courses c
ON s.Course_ID = c.Course_ID;
```

This connects students with their courses.

Q12. What is the difference between INNER JOIN and LEFT JOIN?

Answer:

INNER JOIN returns only rows where there is a match in both tables.

LEFT JOIN returns **all rows from the left table**, even if there is no matching row in the right table.

Example:

Suppose:

Students
1 Rahul
2 Priya
3 Amit

But only Rahul and Priya have courses.

With INNER JOIN:

Rahul
Priya

With LEFT JOIN:

Rahul
Priya
Amit → NULL course

Real-world example:
If you want **all students**, even students who haven't enrolled in a course yet, use LEFT JOIN.

Q13. What is a self join?

Answer:
A **self join is when a table is joined with itself**.

It's useful when records in the same table have relationships with each other.

Example:

An employee table:

Employee_ID | Name | Manager_ID
1 | Rahul | NULL
2 | Priya | 1
3 | Amit | 1

We can use a self join to find each employee's manager.

```
SELECT e.Name AS Employee,
    m.Name AS Manager
FROM employees e
LEFT JOIN employees m
ON e.Manager_ID = m.Employee_ID;
```

Here the employees table is used twice.

Q14. What is a subquery?

Answer:
A **subquery is a query written inside another SQL query**.

Example:

Find employees earning more than the average salary:

```
SELECT name, salary
FROM employees
WHERE salary > (
  SELECT AVG(salary)
  FROM employees
);
```

The inner query calculates the average salary, and the outer query finds employees earning more than it.

Real-world example:
First calculate the average class mark, then find students who scored above that average.

Q15. Subquery vs JOIN?

Answer:
Both can be used to solve many of the same problems, but they work differently.

JOIN combines data from multiple tables.

Subquery uses the result of one query inside another query.

Example:

Suppose we want employee names and their department names.

A JOIN is natural:

```
SELECT e.name, d.department_name
FROM employees e
JOIN departments d
ON e.department_id = d.id;
```

A subquery could also retrieve related information, but a JOIN is often clearer for combining columns from related tables.

Q16. What is a correlated subquery?

Answer:
A **correlated subquery is a subquery that depends on a value from the outer query**. It is evaluated with reference to the current outer row.

Example:

Find employees earning more than the average salary of their own department:

```
SELECT e.name, e.salary, e.department_id
FROM employees e
WHERE e.salary > (
   SELECT AVG(e2.salary)
   FROM employees e2
   WHERE e2.department_id = e.department_id
);
```

The inner query uses:

e.department_id

from the outer query.

That's why it is called **correlated**.

Q17. What are aggregate functions?

Answer:
Aggregate functions perform calculations on **multiple rows and return a single result per group**.

Common aggregate functions are:

```
COUNT()
SUM()
AVG()
MIN()
```

MAX()

Example:

```
SELECT AVG(salary)
FROM employees;
```

This calculates the average salary of all employees.

Or:

```
SELECT department, MAX(salary)
FROM employees
GROUP BY department;
```

This finds the highest salary in each department.

Q18. How do you find the second-highest salary?

Answer:
One simple approach is:

```
SELECT MAX(salary)
FROM employees
WHERE salary < (
  SELECT MAX(salary)
  FROM employees
);
```

The inner query finds the highest salary.

The outer query finds the highest salary that is **less than the highest salary**.

Example:

```
Salary
------
100000
80000
70000
50000
```

Highest = 100000

Second-highest = 80000

Important: This returns the second-highest **distinct** salary.

Q19. How do you find duplicate records?

Answer:
We can use GROUP BY with HAVING.

For example, to find duplicate emails:

```
SELECT email, COUNT(*)
FROM employees
GROUP BY email
HAVING COUNT(*) > 1;
```

Example:

```
Email
----------------
a@gmail.com
b@gmail.com
a@gmail.com
```

The query identifies:

a@gmail.com → 2

because it occurs more than once.

Q20. How do you find employees who earn more than their manager?

Answer:
We can use a **self join** because both employees and managers are stored in the same table.

```
SELECT e.name AS employee,
    e.salary AS employee_salary,
    m.name AS manager,
    m.salary AS manager_salary
FROM employees e
JOIN employees m
ON e.manager_id = m.employee_id
WHERE e.salary > m.salary;
```

Example:

Rahul → Salary 80,000
Manager Priya → Salary 70,000

Rahul will appear because:

80,000 > 70,000

Q21. How do you find the nth-highest salary?

Answer:
There are multiple approaches. A common modern approach is using DENSE_RANK().

```
SELECT salary
FROM (
  SELECT salary,
      DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
  FROM employees
) x
WHERE rnk = 3;
```

This finds the **third-highest distinct salary**.

Example:

100000 → rank 1
90000 → rank 2
80000 → rank 3
70000 → rank 4

So the third-highest salary is 80000.

Q22. What is a window function?

Answer:
A **window function performs a calculation across a set of related rows without combining those rows into a single row**.

That's the important difference from GROUP BY.

Example:

```
SELECT name,
    department,
    salary,
    RANK() OVER (
      PARTITION BY department
      ORDER BY salary DESC
    ) AS salary_rank
FROM employees;
```

This gives every employee a salary rank **within their department**, while still returning every employee row.

Q23. What is the difference between RANK(), DENSE_RANK(), and ROW_NUMBER()?

Answer:
All three assign numbers to rows, but they handle ties differently.

Suppose salaries are:

100000
90000
90000
80000

ROW_NUMBER()

100000 → 1
90000 → 2
90000 → 3
80000 → 4

Every row gets a unique number.

RANK()

100000 → 1
90000 → 2
90000 → 2
80000 → 4

It leaves a gap after a tie.

DENSE_RANK()

100000 → 1
90000 → 2
90000 → 2
80000 → 3

It doesn't leave a gap.

Normalization

Q24. What is normalization?

Answer:
Normalization is the process of organizing data into tables to reduce unnecessary duplication and improve data consistency.

Example:

Instead of storing:

Student | Course | Teacher
Rahul | DBMS | Ravi
Priya | DBMS | Ravi
Amit | DBMS | Ravi

we can separate course information:

Students
Student_ID | Name

Courses
Course_ID | Course_Name | Teacher

This avoids repeatedly storing the same course information.

Q25. Why do we normalize databases?

Answer:
We normalize databases mainly to:

- Reduce data duplication.
- Avoid inconsistent data.
- Make updates easier.
- Reduce storage in many cases.
- Improve data organization.

Example:
If a teacher's name is stored in 100 student records, changing the teacher's name requires many updates.

If the teacher information is stored once in a separate table, we only need to update it in one place.

Q26. What is 1NF?

Answer:
A table is in **First Normal Form (1NF)** when each column contains **atomic/single values**, rather than multiple values in one cell.

Bad design:

Student | Phone
Rahul | 9876, 8765

The phone column contains multiple values.

Better:

Student | Phone
Rahul | 9876
Rahul | 8765

Each cell contains one value.

Q27. What is 2NF?

Answer:
A table is in **Second Normal Form (2NF)** when:

1. It is already in 1NF.
2. Every non-key attribute depends on the **whole primary key**, not just part of it.

This mainly matters when we have a **composite key**.

Example:

Suppose:

Student_ID + Course_ID → Primary Key

Student_ID | Course_ID | Student_Name

Student_Name depends only on Student_ID, not on the complete (Student_ID, Course_ID) key.

So we separate the student information into another table.

Q28. What is 3NF?

Answer:
A table is in **Third Normal Form (3NF)** when:

1. It is in 2NF.
2. Non-key attributes do not depend on other non-key attributes.

In simple terms, **non-key columns should depend on the key, the whole key, and nothing but the key.**

Example:

Employee_ID | Department_ID | Department_Name

Department_Name depends on Department_ID, not directly on Employee_ID.

So department information should be separated into another table.

Q29. What is BCNF?

Answer:
BCNF (Boyce-Codd Normal Form) is a stronger version of 3NF.

A table is in BCNF if **every determinant is a candidate key**.

In other words, whenever one attribute determines another attribute, the determining attribute must be a candidate key.

Example:
Consider a table where a non-key attribute determines another attribute. If that determining attribute isn't a candidate key, the table may violate BCNF.

Q30. What is denormalization?

Answer:
Denormalization is intentionally adding some redundancy to a database to improve read performance or simplify queries.

It is basically the opposite direction of normalization.

Example:

Instead of joining:

Orders
Customers

Products

every time we display an order, we might store some frequently needed customer information directly in the order-related data.

This can make reads faster, but creates duplicated data that must be kept consistent.

Q31. When would you denormalize?

Answer:
We may denormalize when:

- Read performance is very important.
- Complex joins are becoming expensive.
- The data is read much more frequently than it is updated.
- We want to reduce the number of joins for frequently used queries.

Example:
An e-commerce website may frequently display:

Order ID
Customer Name
Product Name
Price

If generating this information requires many expensive joins millions of times, some carefully chosen redundant data may improve read performance.

Transactions & Concurrency

Q32. What is a transaction?

Answer:
A **transaction is a sequence of database operations treated as one logical unit of work.**

Either the whole transaction succeeds, or the database rolls it back so the intended changes aren't partially applied.

Example: Bank transfer

Transfer ₹1,000 from Rahul to Priya:

1. Deduct ₹1,000 from Rahul
2. Add ₹1,000 to Priya

Both operations should succeed together.

If money is deducted from Rahul but isn't added to Priya, the transaction should be rolled back.

Q33. Explain ACID properties.

Answer:
ACID describes important properties of database transactions.

A — Atomicity

All operations happen, or none happen.

Example:

Debit Rahul
Credit Priya

Both happen, or both are rolled back.

C — Consistency

The transaction should leave the database in a **valid state according to its rules and constraints**.

Example:

A transaction shouldn't violate a rule such as a balance constraint.

I — Isolation

Multiple transactions running at the same time should not improperly interfere with each other.

Example:

Two people shouldn't see incorrect intermediate results from each other's transactions.

D — Durability

Once a transaction is successfully committed, its changes should survive failures such as a system crash, subject to the database's durability guarantees.

Q34. What is concurrency control?

Answer:
Concurrency control is the set of techniques used to manage multiple transactions running at the same time while maintaining database correctness.

Example:
Two people try to book the **last available seat** at the same time.

Without proper concurrency control, both might successfully book the same seat.

The database needs mechanisms such as locking or MVCC to prevent incorrect results.

Q35. What is a dirty read?

Answer:
A **dirty read occurs when one transaction reads data that another transaction has changed but not yet committed**.

If the first transaction later rolls back, the value that was read never actually became permanent.

Example:

Transaction A:
Balance = ₹10,000
Changes it to ₹5,000
But hasn't committed.

Transaction B:
Reads ₹5,000

Then Transaction A rolls back.

The real balance returns to ₹10,000.

Transaction B had read **uncommitted/dirty data**.

Q36. What is a non-repeatable read?

Answer:
A **non-repeatable read occurs when a transaction reads the same row twice and gets different values because another transaction modified and committed that row between the two reads.**

Example:

Transaction A reads:

Balance = ₹10,000

Transaction B changes it to:

₹8,000

and commits.

Transaction A reads again and gets:

₹8,000

The same query within Transaction A produced different values.

That's a **non-repeatable read**.

Q37. What is a phantom read?

Answer:
A **phantom read occurs when a transaction executes the same query twice and finds a different set of rows because another transaction inserted, deleted, or otherwise changed rows matching the query.**

Example:

Transaction A asks:

```
SELECT * FROM employees
WHERE salary > 50000;
```

It gets:

Rahul
Priya

Then Transaction B inserts another employee with salary 60,000 and commits.

Transaction A runs the same query again and gets:

Rahul
Priya
Amit

Amit is the **phantom row**.

Q38. What are isolation levels?

Answer:
Isolation levels define how much one transaction is protected from changes made by other concurrent transactions.

The commonly discussed SQL isolation levels are:

READ UNCOMMITTED
READ COMMITTED
REPEATABLE READ
SERIALIZABLE

From weaker to stronger isolation:

READ UNCOMMITTED
↓
READ COMMITTED
↓
REPEATABLE READ
↓
SERIALIZABLE

Generally, stronger isolation provides more protection but can reduce concurrency/performance.

Example:

Imagine two people editing the same document.

- Weak isolation → they may see each other's intermediate changes.
- Strong isolation → one person's work is more isolated until appropriate synchronization occurs.

Indexing

Q39. What is a database index?

Answer:
A **database index is a data structure that helps the database find rows faster without scanning the entire table**.

It is similar to an index in a textbook.

Example:

Suppose a table has:

1 million employees

and we search:

```
SELECT *
FROM employees
WHERE employee_id = 500123;
```

Without a suitable index, the database may need to examine many rows.

With an index on employee_id, it can locate the required row much more efficiently.

Q40. How does an index improve performance?

Answer:
An index creates an additional data structure that allows the database to **locate matching rows efficiently**, rather than scanning every row.

Many relational databases commonly use **B-tree/B+tree-like structures** for indexes, while other index types exist for specialized workloads.

Example:

Think about a textbook with 1,000 pages.

Without an index:

> "Find every page mentioning DBMS."

You might scan every page.

With an index:

> Look at the index → find "DBMS" → jump to the relevant pages.

That's the basic idea of a database index.

Q41. When can an index hurt performance?

Answer:
Indexes improve reads, but they aren't free.

They can hurt performance because:

1. **INSERT becomes more expensive** — the index also needs to be updated.

2. **UPDATE can become more expensive** — especially when indexed columns change.
3. **DELETE can become more expensive** — index entries may need to be removed.
4. **Indexes consume additional storage.**
5. Too many indexes can increase maintenance overhead.

Example:

Suppose an e-commerce table has:

10 million orders

and you create indexes on 20 different columns.

Searching might become faster for some queries, but every new order may require maintaining many index structures.

⭐ DBMS Questions You Should Be Especially Strong At

For placements, I'd make these **non-negotiable**:

1. Primary Key vs Foreign Key
2. Primary Key vs Unique Key
3. Candidate Key vs Composite Key
4. WHERE vs HAVING
5. GROUP BY
6. All JOIN types
7. INNER JOIN vs LEFT JOIN
8. Subquery
9. Correlated Subquery
10. Second-highest salary
11. Duplicate records
12. Nth-highest salary
13. Window Functions
14. RANK vs DENSE_RANK vs ROW_NUMBER
15. Normalization
16. 1NF, 2NF, 3NF
17. Transactions
18. ACID
19. Concurrency
20. Dirty / Non-repeatable / Phantom reads
21. Isolation levels
22. Indexes

23. Why indexes improve performance
24. When indexes hurt performance

And one particularly important thing for **MCA fresher interviews**: don't prepare SQL as definitions only. You should actually be able to sit in front of an interviewer and **write queries from scratch**.

A good target is being comfortable with queries involving:

SELECT → WHERE → GROUP BY → HAVING → JOIN → subquery → CTE → window functions

because interviewers can start with a simple query and progressively make it harder.